SLO-02: Defining SLIs

> **Series:** SLO — Service Level Objectives | **Notebook:** 2 of 6 | **Created:** June 2026 | **Last Updated:** 10/14/2026

Overview

An SLI is a `good ÷ total` ratio expressed as a percentage. This notebook builds the four SLI shapes you will use most — availability, latency, error rate, and custom business SLIs — as DQL queries. Each SLI comes in two forms: a **notebook read-out** that collapses the result to one number you can read, and an **SLO-ready form** that keeps `sli` as a timeseries array, which is what the SLO app requires. Every query here was validated against a live tenant.

The pattern is always the same: count the *good* events, count the *total* events, divide, multiply by 100. The skill is in defining "good" correctly for each indicator.

Table of Contents

1. The SLI Ratio Pattern
2. Availability SLI
3. Latency SLI — Two Approaches
4. Error-Rate SLI
5. Custom / Business SLIs
6. Validate Before You Commit

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS Gen3 with Grail
Permissions	`storage:metrics:read`, `storage:spans:read`, `storage:bizevents:read` as applicable
Prior reading	SLO-01 (fundamentals)
MCP (optional)	Dynatrace MCP server to verify queries before adding them to an SLO

1. The SLI Ratio Pattern

Every SLI reduces to:

```
SLI % = (good events ÷ total events) × 100
```

In Dynatrace you express this as a DQL query that returns a field named `sli`. The SLO app requires `sli` to be an **array** of doubles (a timeseries), not a single number, and evaluates it over the SLO's own window. Two query families serve different data:

- **Metric-based** (`timeseries`) — fast, pre-aggregated, ideal for service request metrics. Good and total are sums of metric series; divide element-wise with `[]`.
- **Event-based** (`fetch spans` / `logs` / `bizevents`) — flexible, lets you define "good" with any filter using `countIf()`. Higher query cost but expresses conditions a metric cannot. For an SLO, aggregate with `makeTimeseries` rather than `summarize`, so `sli` stays an array.

Both are valid SLI sources. Reach for metric-based when a service metric already captures the condition; reach for event-based when "good" needs a predicate the metrics do not carry.

> **Sources:** Create service-level objectives (DT docs) — *"must include an sli field that returns an array of double type values"*; *"use the makeTimeseries command for deriving the sli field"*.

2. Availability SLI

Availability = the proportion of requests that did **not** fail. Using the pre-aggregated service request metrics, good = `total − failures`:

```
// Availability SLI — metric-based, request success ratio
// Re-validated on a live tenant 10/14/2026: returned 96.21% over 30 d.
// sli divides the window's successes by its requests. Averaging the daily
// ratios (arrayAvg) instead weights a quiet day like a busy one: 96.18% here
// interval:24h, not 1d — a calendar duration here raises
// DEPRECATED_DURATION_UNIT_DAYS and is silently rewritten to 24h anyway
timeseries {
  total = sum(dt.service.request.count),
  failures = sum(dt.service.request.failure_count)
}, from:-30d, interval:24h
| fieldsAdd sli = ((arraySum(total) - arraySum(failures)) / arraySum(total)) *
100
| fields sli, total, failures
```

The query above is the notebook read-out: it divides the window's total successes by its total requests, so one `sli` number covers the whole window. Averaging the daily ratios with `arrayAvg` would weight a quiet day as heavily as a busy one. The SLO app cannot use a scalar `sli` — it needs the array, and it applies its own window. For an SLO, use the SLO-ready form below and remove `from:` (and any `interval:`) when you paste it (SLO-05).

To scope the SLI to one service, filter on `dt.smartscape.service` and convert the ID with `toSmartscapeId()`: `filter: {dt.smartscape.service == toSmartscapeId("SERVICE-...")}`. Or split `by: {dt.smartscape.service}`, as below. `dt.entity.service` still works, but it is `deprecated`. Comparing `dt.smartscape.service` to a plain string returns no rows, and Grail raises only a `SMARTSCAPEID_TO_STRING_COMPARISON` warning.

> **Dictionary:** `dt.smartscape.service` (`stable`, type `smartscapeId`), `dt.entity.service` (`deprecated`), read 10/12/2026. The string comparison was run against the validation tenant 10/12/2026: 0 rows plus the warning, where the `toSmartscapeId()` form returned the service's requests.

```
// SLO-ready availability SLI — sli stays an array, one series per service
// Validated 10/12/2026: 48 service series, notifications []
// Remove from: when pasting into an SLO — the SLO sets its own window
timeseries {
  total = sum(dt.service.request.count),
  failures = sum(dt.service.request.failure_count)
}, by: { dt.smartscape.service }, from:-24h
| fieldsAdd sli = ((total[] - failures[]) / total[]) * 100
```

3. Latency SLI — Two Approaches

Latency has no single canonical SLI in Dynatrace, because there is no built-in "count of requests faster than X" metric. There are two valid approaches — pick by what you actually need.

Decision

Approach	What it measures	Use when	Trade-off
Percentile threshold	Is the p95/p99 under a limit?	You care about the tail of the distribution	A threshold check, not a true good/total ratio — one number per window
Good/total ratio	What fraction of requests beat the limit?	You want a real SLI percentage and an error budget in "slow requests"	Computed from spans (event-based) — higher query cost

3a. Percentile-threshold (metric-based)

The p95 response time, in milliseconds (`dt.service.request.response_time` is in microseconds, so divide by 1000). Do **not** add `rollup:` here. `dt.service.request.response_time` is a histogram metric, and `percentile` takes `rollup:` only for gauge or count metrics. With `rollup: avg`, each series is averaged first and the query returns the p95 of those averages: 5,835 ms, against 149 ms without it, on the validation tenant (10/12/2026). Nothing warns.

`arrayAvg` of hourly p95 values is a trend figure, not the p95 of the day. For a single-window p95, drop `interval:`.

> **Sources:** DQL metric commands (DT docs) — `percentile` *"applies to metrics of type histogram"*, and for a gauge or count *"the rollup parameter needs to be specified"*.

```
// Latency — p95 response time in ms (metric-based, histogram metric)
// No rollup: — rollup:avg would return the p95 of per-series averages
// (5,835 ms vs 149 ms on the validation tenant, 10/12/2026)
timeseries p95 = percentile(dt.service.request.response_time, 95),
  from:-24h, interval:1h
| fieldsAdd p95_ms = p95[] / 1000.0
| fieldsAdd p95_hourly_avg_ms = arrayAvg(p95_ms)
| fields p95_hourly_avg_ms, p95_ms
```

3b. Good/total ratio (span-based)

A true latency SLI: the fraction of requests that completed under the threshold. `countIf()` defines "good" with a duration predicate.

Count each request once, at its request root span. A `span.kind == "server"` filter also counts server spans nested inside a request, so it inflates the denominator: on the validation tenant (last hour, 10/14/2026) 13.8% of server spans were nested, and the server-span form read 98.83% where the request-root form read 98.16%. The trace semantic model marks `request.is_root_span` as deprecated, *"Replaced by transaction.is_root_span in combination with transaction.is_endpoint_request"*. Both replacement fields are `experimental`. On the validation tenant the pair matched `request.is_root_span` span for span, and it counted 136,713 requests where `dt.service.request.count` counted 136,671 in a back-to-back run over the same hour. Request roots also include non-server spans such as message consumers, as the metric does.

```
// Latency SLI — span-based good/total ratio under 500ms, one span per request
// Validated on a live tenant 10/14/2026: returned 98.16% (134,318/136,836)
// A 1h window on live traffic moves — treat the figure as a shape check,
// not a number to reproduce
fetch spans, from:-1h
| filter transaction.is_root_span == true and transaction.is_endpoint_request
== true
| summarize {
    good = countIf(duration &lt; 500ms),
    total = count()
  }
| fieldsAdd sli_latency = (good * 100.0) / total
| fields sli_latency, good, total
```

> **Note the field values:** `duration` compares directly against a duration literal (`500ms`) — no nanosecond constants. If you filter on `span.kind` , its values are lowercase (`"server"` , `"client"`). Both are common mistakes that silently return zero rows.
>
> **Sources:** Traces — semantic dictionary (DT docs) — `request.is_root_span` : *"Replaced by transaction.is_root_span in combination with transaction.is_endpoint_request"*; page updated Sep 21, 2026, read 10/14/2026. **Dictionary:** no `transaction.*` row in the validation tenant's `dt.semantic_dictionary.fields` yet, read 10/14/2026 (control: `startsWith(name, "request.")` → `request.is_root_span` (`deprecated`), `request.is_failed` (`deprecated`)); the docs page is newer, and both fields are populated on spans there.

The query above is the notebook read-out. The SLO-ready form uses `makeTimeseries` instead of `summarize` , so `sli` stays an array:

```
// SLO-ready latency SLI — makeTimeseries, as the SLO docs require for event
data
// Validated 10/14/2026: 13 five-minute buckets, 96.6–99.7%, notifications []
// Remove from: and interval: when pasting into an SLO
fetch spans, from:-1h
| filter transaction.is_root_span == true and transaction.is_endpoint_request
== true
| makeTimeseries { good = countIf(duration &lt; 500ms), total = count() },
interval:5m
| fieldsAdd sli = (good[] * 100.0) / total[]
```

4. Error-Rate SLI

Error rate is availability's complement — sometimes you want to track it directly (and set the SLO as "error rate ≤ X%"). The good/total framing still applies; here "good" stays the success count, and you report the failure ratio alongside:

```
// Error-rate SLI — failure ratio, metric-based
// Validated 10/14/2026: 3.13% over 7 d (arrayAvg of the hourly ratios read
3.16%)
timeseries {
  total = sum(dt.service.request.count),
  failures = sum(dt.service.request.failure_count)
}, from:-7d, interval:1h
| fieldsAdd error_rate = arraySum(failures) / arraySum(total) * 100
| fields error_rate
```

Express the SLO target as `success ≥ 99.5%` rather than `errors ≤ 0.5%` where you can — the SLO app and the error-budget math (SLO-03) are oriented around the "good" percentage, and keeping every SLI in the same direction avoids inverted-threshold confusion. For the SLO itself, that means the SLO-ready availability form in §2. The `error_rate_avg` scalar above is a notebook read-out.

5. Custom / Business SLIs

The modern SLO app accepts any `good ÷ total` over any Grail data — which means business outcomes become first-class SLIs. Checkout success, order completion, or payment authorization rate computed from business events:

```
// Business SLI — checkout success rate from business events
// Adapt event.type / field names to your bizevent schema
fetch bizevents, from:-24h
| filter event.type == "checkout"
| summarize {
    good = countIf(checkout.result == "success"),
    total = count()
  }
| fieldsAdd sli_checkout = (good * 100.0) / total
| fields sli_checkout, good, total
```

Business SLIs are where SLOs earn executive attention — "99.9% of checkouts completed" lands in a way a service availability number does not. See the BIZEV series

for building the business-event stream this query consumes.

The query above is the notebook read-out. The SLO-ready form uses `makeTimeseries`:

```
// SLO-ready business SLI — makeTimeseries keeps sli as an array
// Adapt event.type / field names to your bizevent schema
// Remove from: and interval: when pasting into an SLO
fetch bizevents, from:-24h
| filter event.type == "checkout"
| makeTimeseries { good = countIf(checkout.result == "success"), total =
count() }, interval:1h
| fieldsAdd sli = (good[] * 100.0) / total[]
```

6. Validate Before You Commit

An SLI query that returns nothing — wrong field name, wrong `span.kind` case, a filter that matches no data — produces an SLO that silently never computes. Before you save an SLI into an SLO:

1. **Run it in a notebook** over a representative window and confirm it returns a sane percentage.
2. **Check the denominator.** A `total` of zero means the filter is too narrow; the SLI will be undefined.
3. **Verify with the MCP server** (`execute-dql`) against the live tenant if you have it wired up.
4. **Only then** paste it into the SLO definition.

This notebook-as-scratchpad discipline is the single best defence against a dashboard full of SLOs that read "no data."

> **Sources:** Service-Level Objectives (DT docs), DQL — timeseries and countIf (DT docs). All five queries re-executed against a live tenant 08/28/2026 (the bizevents example returns 0 rows there by design — it is a schema template, not a tenant claim). The availability query's `interval` was changed from `1d` to `24h` in the same pass: a calendar duration raises `DEPRECATED_DURATION_UNIT_DAYS` and is silently rewritten to `24h`, so the query was never running on the interval it appeared to specify. On 10/12/2026 the p95 query (§3a) lost its `rollup: avg`, and the three SLO-ready forms were added. All were executed against the live tenant that day. The bizevent form returns 0 rows there, like its read-out twin; the same `makeTimeseries` shape

> returned 25 hourly buckets over all bizevents. On 10/14/2026 the availability and error-rate read-outs switched from `arrayAvg` of per-bucket ratios to window totals, and both span-based latency SLIs from `span.kind == "server"` to request root spans; all four were re-executed that day.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.